WAB Exposé

Provadis School of International Management and Technology

Analyzing the Correlation Between Cyclomatic Complexity and Code Quality in Highly Instrumental Python Projects

Rubin Chempananickal James
rubin.chempananickal-james@stud-provadis-hochschule.de
Matriculation Number: D876

Department: Information Technology
Module: Agile Softwareentwicklung und Softwaretechnik
Reviewer: Philip Ceh

05.05.2026

Contents

Exposé

1. Introduction

Python[1] is one of the most popular programming languages in the world[2], with a vast ecosystem of third-party packages available through the Python Package Index (PyPI)[3]. As the use of Python continues to grow, and especially as the frequency of commits to these packages by Large Language Models (LLMs) increases, understanding the quality and maintainability of these packages becomes increasingly important for developers and organizations that rely on them.

Cyclomatic complexity is a graph theory based approach to compute the number of independent paths through a function[4]. This gives a rough estimate of how many separate test cases one might need to cover every output of that function. A function with a higher cyclomatic complexity is traditionally less desirable, as it would theoretically have a larger number of edge cases wherein a potential bug could hide.

2. Objectives

The objective of this paper is to analyze the relationship between cyclomatic complexity and bug-introducing commits in Python packages on PyPI, using the Sliwezki-Zeller-Zimmermann (SZZ)[5] algorithm to identify bug-introducing commits. By mining a representative sample of popular Python packages, the aim is to understand whether higher complexity functions are more likely to be associated with bugs, and how this relationship has evolved over time.

3. Background and Related Work

Previous research has explored the relationship between code complexity and software defects in various programming languages[6–8], but there is a lack of comprehensive studies focused specifically on Python packages in

the PyPI ecosystem. Additionally, while the SZZ algorithm has been widely used to identify bug-introducing commits, its application in the context of Python packages and its relationship with cyclomatic complexity does not appear to have been thoroughly investigated yet.

4. Research Questions

The paper will seek to answer the following research questions:

1. Can a meaningful correlation be observed between cyclomatic complexity and bug-proneness in Python packages on PyPI?
2. How do bug-fixing commits affect the cyclomatic complexity of functions? Do they tend to increase, decrease, or stay the same?

5. Methodology

The representative dataset will be the most depended upon Python packages on PyPI. This ranking will be extracted from Libraries.io, a platform by Sonar that tracks dependencies and usage across various package ecosystems[9]. Starting from the top of this ranking, packages will be considered in order until ten packages with resolvable GitHub source repositories could be identified. For each selected package, release metadata and repository information will then be verified against the corresponding PyPI project metadata[3]. Afterwards, the repositories of the selected packages will be mined for data analysis[10].

For each package, bug fixing commits will be found by searching within commit messages that allude to fixing a bug. The SZZ algorithm will then be applied to identify the bug-introducing commits that led to these fixes. For each function within the codebase which had at least one bug fix, the cyclomatic complexity will be computed at each commit where it was modified. The change in cyclomatic complexity across bug-fixing commits will then be tracked and analyzed.

6. Planned Structure

The paper will likely be structured as follows:

- Introduction
- Background and Related Work
- Methodology
- Results
- Discussion
- Conclusion and Future Work
- References
- AI Declaration
- Declaration of Authorship

References

1. Van Rossum G, Drake FL. *Python 3 Reference Manual*. CreateSpace; 2009.

2. Chaudhary P, Agrawal L, Ali A. Modern programming languages–characteristics and recommendations for instruction. *Issues in Information Systems.* 2025;26(2).

3. Python Software Foundation. Python Package Index (PyPI). Accessed May 4, 2026. https://pypi.org/

4. McCabe TJ. A Complexity Measure. *IEEE Transactions on Software Engineering.* 1976;(4):308-320. doi:10.1109/TSE.1976.233837

5. Sliwerski J, Zimmermann T, Zeller A. When Do Changes Induce Fixes?. In: *Proceedings of the 2005 International Workshop on Mining Software Repositories.* 2005:1-5. doi:10.1145/1083142.1083147

6. Bachmann A, Bird C, Rahman F, Devanbu P, Bernstein A. The Missing Links: Bugs and Bug-Fix Commits. In: *Proceedings of the Eighteenth ACM SIGSOFT International Symposium on Foundations of Software Engineering.* 2010:97-106. doi:10.1145/1882291.1882308

7. Hassan AE. Predicting Faults Using the Complexity of Code Changes. In: *2009 IEEE 31st International Conference on Software Engineering.* 2009:78-88. doi:10.1109/ICSE.2009.5070510

8. Nagappan N, Ball T. Use of Relative Code Churn Measures to Predict System Defect Density. In: *Proceedings of the 27th International Conference on Software Engineering.* 2005:284-292. doi:10.1109/ICSE.2005.1553571

9. SonarSource Sàrl. Libraries.io. Accessed May 4, 2026. https://libraries.io/pypi

10. Kalliamvakou E, Gousios G, Blincoe K, Singer L, German DM, Damian D. An In-Depth Study of the Promises and Perils of Mining GitHub. *Empirical Software Engineering.* 2016;21(5):2035-2071. doi:10.1007/s10664-015-9393-5

11. Python Packaging Authority. Python Packaging User Guide. Accessed May 4, 2026. https://packaging.python.org/